

Trabajo Práctico 2

Programación Lógica

Paradigmas de Lenguajes de Programación
1^{er} cuatrimestre 2026

Fecha de entrega: 3 de julio

Razonando con lenguajes naturales

El objetivo de este trabajo práctico es analizar sintácticamente un pequeño fragmento del idioma español para luego darle significado. A partir de dicho análisis, buscaremos utilizar fórmulas de la lógica proposicional para representar a la semántica a asignarle y realizar algunas inferencias lógicas sobre estas fórmulas.

El lenguaje y su semántica

Para simplificar el proceso de darle semántica al lenguaje, analizaremos solo un fragmento muy reducido del idioma. La gramática con la que trabajaremos es la siguiente:

Frase	→	Condición	Oración	Consecuente	Oración
Frase	→	Oración			
Oración	→	Sujeto	Predicado		
Sujeto	→	DeterminanteFem	SustantivoFem		
Sujeto	→	DeterminanteMasc	SustantivoMasc		
Sujeto	→	PronombrePersonal			
Predicado	→	VerboTrans	Preposición	Sujeto	
Predicado	→	Negación	VerboTrans	Preposición	Sujeto
Predicado	→	VerboIntrans			
Predicado	→	Negación	VerboIntrans		

Esta gramática define las construcciones básicas con las que se va a trabajar. El vocabulario del lenguaje va a estar conformado por los símbolos terminales de la gramática. Por ejemplo, un posible vocabulario sería:

SustantivoMasc	→	alfajor iglú pitufo
SustantivoFem	→	persona gaviota mermelada
DeterminanteFem	→	una
DeterminanteMasc	→	un
PronombrePersonal	→	alfonso matilde charlyGarcía doctorDoom
VerboTrans	→	hechiza molesta admira
Preposición	→	a
VerboIntrans	→	vuela estudia come
Negación	→	no niAhí
Condición	→	si cuando
Consecuente	→	entonces luego

Entonces, algunas de las frases que pertenecen a esta gramática son:

si un pitufo admira a una mermelada entonces doctorDoom vuela
alfonso no hechiza a una gaviota
charlyGarcía estudia
un iglú no molesta a matilde

La semántica de cada frase va a estar representada mediante una fórmula de la lógica proposicional sin disyunciones, cuya sintaxis se define de la siguiente manera:

$$\tau ::= x \mid \tau \wedge \tau \mid \neg \tau \mid \tau \Rightarrow \tau$$

La forma de darle semántica a una frase es la siguiente:

- Las variables proposicionales son representadas por los sustantivos, pronombres y verbos.
- Todas las fórmulas proposicionales están conectadas por conjunciones, salvo en el caso de la construcción gramatical con condicional:

Frase $\rightarrow$ Condicional Oración Consecuente Oración

en donde la fórmula correspondiente a la primera oración está conectada con la correspondiente a la segunda con una implicación.

- Cuando aparece una **Negación** en la gramática, la fórmula subsiguiente se representa negada.
- Los determinantes y preposiciones no afectan la representación semántica.

Por ejemplo, la siguiente tabla muestra algunas frases, junto con su semántica.

Frase	Semántica
charlyGarcía estudia	<i>charlyGarcía</i> $\wedge$ <i>estudia</i>
un pitufo no vuela	<i>pitufo</i> $\wedge$ $\neg$ <i>vuela</i>
alfonso no hechiza a una gaviota	<i>alfonso</i> $\wedge$ $\neg$ (<i>hechiza</i> $\wedge$ <i>gaviota</i>)
si una mermelada vuela entonces matilde no come	<i>mermelada</i> $\wedge$ <i>vuela</i> $\Rightarrow$ <i>matilde</i> $\wedge$ $\neg$ <i>come</i>
cuando doctorDoom no vuela entonces un pitufo molesta a charlyGarcía	<i>doctorDoom</i> $\wedge$ $\neg$ <i>vuela</i> $\Rightarrow$ <i>pitufo</i> $\wedge$ (<i>molesta</i> $\wedge$ <i>charlyGarcía</i>)

Para llevar a cabo esta tarea en Prolog, una frase va a estar representada por una lista de palabras. La fórmula que representa la semántica asociada va a ser construida mediante los constructores `&`, `>` y `~`, para representar a la conjunción, la implicación y la negación respectivamente. Para hacer más legible la fórmula, estos constructores de fórmulas van a ser definidos como operadores, de la siguiente manera:

```
:- op(900, yfx, >).           % implicación
:- op(800, yfx, &).           % conjunción
:- op(750, fy, ~).            % negación
```

Notar que al estar definida la precedencia de cada operador, pueden obviarse los paréntesis en una fórmula siempre y cuando esto no modifique el significado de la misma.

Ejercicio 1

Utilizando esta representación, se debe implementar en Prolog el predicado `parsea(-Frase,-Semántica)`, que es verdadero cuando **Frase** es una frase construida a partir de las reglas gramaticales ya definidas, y **Semántica** es la fórmula que representa su significado asociado. Por ejemplo, si evaluamos la siguiente frase (el orden entre los resultados puede ser otro, en función del orden en la definición de las reglas gramaticales):

```
?- parsea(Frase, Semántica).
```

```
⋮
```

```
Frase = [si, una, persona, hechiza, a, una, gaviota, entonces, un, alfajor, hechiza, a, una, mermelada]
Semántica = persona& (hechiza&gaviota)>alfajor& (hechiza&mermelada)
```

```
⋮
```

```
Frase = [si, una, persona, hechiza, a, una, gaviota, entonces, un, alfajor, hechiza, a, un, iglú]
```

```

Semántica = persona& (hechiza&gaviota)>alfajor& (hechiza&iglú)
:
:
Frase = [si, una, persona, hechiza, a, una, gaviota, entonces, un, alfajor, hechiza, a, un, pitufo]
Semántica = persona& (hechiza&gaviota)>alfajor& (hechiza&pitufufo)
:
:
Frase = [matilde, no, hechiza, a, una, gaviota]
Semántica = matilde& ~(hechiza&gaviota)
:
:

```

Tener en cuenta que los operadores $\&$, $>$ y $\sim$ no son más que constructores de términos, que son definidos infijos por comodidad. Por ejemplo, la fórmula `matilde& ~(hechiza&gaviota)` puede ser reescrita de forma normal como `&(matilde, ~(hechiza,gaviota))`.

Ayuda: para realizar el parseo, la gramática así definida tiene buenas propiedades que pueden ser utilizadas. Es posible recorrer la frase (la lista de palabras) de izquierda a derecha, y decidir cuál es la regla que se debe utilizar observando solo la palabra actual en dicha lista.

Razonamiento lógico

Una vez obtenida la semántica de una frase, nos va a resultar útil poder realizar inferencias y chequeos lógicos sobre ella. Por ejemplo, vamos a querer identificar cuándo dos frases son contradictorias, o analizar cuándo una frase puede derivarse lógicamente de otra. Los predicados que se piden definir en esta sección nos van a permitir resolver estos problemas, además de otros más generales. A partir de este punto nos concentraremos en predicados que operan directamente sobre las fórmulas que representan la semántica de las frases.

Ejercicio 2

Definir el predicado `variables(+Fórmula,-Variables)`, el cual es verdadero cuando `Variables` es la lista de variables proposicionales que aparecen en `Fórmula`, sin repetidos.

Por ejemplo,

```

?- variables(persona & (hechiza & gaviota) > alfajor & (hechiza & mermelada), V).
V = [persona, hechiza, gaviota, alfajor, mermelada] ;
false.

```

Ayuda: Puede ser útil el predicado `atom/1`.

Valuaciones

Las valuaciones son mapeos de variables proposicionales a valores de verdad. En este caso representamos el valor verdadero como `t` y el falso como `f`, y representamos las valuaciones como listas de tuplas (`Variable`, `Valor`).

Ejercicio 3

Definir el predicado `valuación(+Variables, -Valuación)`, el cual debe instanciar en `Valuación` todas las valuaciones posibles para las variables en la lista `Variables`.

Por ejemplo,

```

?- valuación([mermelada, gaviota], Valuación).
Valuación = [(mermelada, t), (gaviota, t)] ;
Valuación = [(mermelada, t), (gaviota, f)] ;
Valuación = [(mermelada, f), (gaviota, t)] ;
Valuación = [(mermelada, f), (gaviota, f)].

```

Ejercicio 4

Definir el predicado `esVálida(+Fórmula, +Valuación)`, el cual es verdadero cuando `Valuación` asigna valores de verdad a todas las variables proposicionales de `Fórmula` y bajo esta valuación la fórmula es verdadera.

Por ejemplo,

```
?- esVálida(persona & (hechiza & gaviota), [(persona, f), (hechiza, t), (gaviota, f)]).
false.
```

Ejercicio 5

Definir el predicado `modelo(+Fórmula, -Valuación)`, el cual instancia en `Valuación` todos los modelos para `Fórmula`, es decir, las valuaciones para las cuales la fórmula es válida.

Por ejemplo,

```
?- modelo(gaviota > mermelada, Modelo).
Modelo = [(gaviota, t), (mermelada, t)] ;
Modelo = [(gaviota, f), (mermelada, t)] ;
Modelo = [(gaviota, f), (mermelada, f)] ;
```

Ejercicio 6

Definir el predicado `mejorModelo(+Fórmula, -Valuación)`, el cual instancia en `Valuación` todos los modelos de `Fórmula` que tienen la máxima cantidad de variables verdaderas.

Por ejemplo,

```
?- mejorModelo(~a > a, M).
M = [(a, t)] ;
false.

?- mejorModelo(~(~a & b) > (a & ~b), M).
M = [(a, t), (b, f)] ;
M = [(a, f), (b, t)] ;
false.
```

Ayuda: puede ser útil definir el predicado `cantidadDeVariablesVerdaderas(+Modelo, -Cantidad)` que instancie en `Cantidad` la cantidad de variables verdaderas en `Modelo`.

Ejercicio 7

Definir los siguientes predicados:

- `subvaluación(+Valuación1, +Valuación2)`, que es verdadero cuando `Valuación1` está contenida en `Valuación2`.

Por ejemplo,

```
?- subvaluación([(gaviota, t)], [(gaviota, t), (mermelada, f)]).
true.

?- subvaluación([(gaviota, t)], [(gaviota, f), (mermelada, f)]).
false.
```

- `extiende(+Valuación1, +Fórmula, -Valuación2)`, que es verdadero cuando `Valuación2` es un modelo de `Fórmula` y `Valuación1` es una subvaluación de este modelo.

Por ejemplo,

```
?- extiende([(a,t)], ~a > b, Valuación).
Valuación = [(a, t), (b, t)] ;
```

```

Valuación = [(a, t), (b, f)] ;
false.

?- extiende([(a,f)], ~a > b, Valuación).
Valuación = [(a, f), (b, t)] ;
false.

```

Ejercicio 8

¿El predicado `subvaluación/2` implementado en el ejercicio anterior, es reversible respecto de su primer argumento? Justificar.

Ejercicio 9

Definir los siguientes predicados:

- `consecuencia(+F1, +F2)`, que es verdadero cuando `F2` es consecuencia lógica de `F1`. Una fórmula es consecuencia lógica de otra cuando admite todos sus modelos.
- `equivalentes(+F1, +F2)`, que es verdadero cuando `F1` y `F2` son equivalentes, es decir que sus modelos coinciden.
- `contradictorias(+F1, +F2)`, que es verdadero cuando `F1` y `F2` son contradictorias, es decir que no comparten modelos.
- `modeloÚnico(+F, -Valuación)`, que es verdadero cuando `Valuación` es el único modelo de `F`.

Pautas de Entrega

Importante: se espera que la elaboración de este trabajo sea 100 % de los estudiantes del grupo que realiza la entrega. Así que, más allá de que pueden tomar información de lo visto en las clases o consultar información en la documentación de SWI-Prolog u otra disponible en Internet, no se podrán utilizar herramientas para generar parcial o totalmente en forma automática la resolución del TP (e.g., ChatGPT, Copilot, etc). En caso de detectarse esto, el trabajo será considerado como un plagio, por lo que será gestionado de la misma forma que se resuelven las copias en los parciales u otras instancias de evaluación.

Cada predicado asociado a los ejercicios debe contar con ejemplos (tests) que muestren que exhibe la funcionalidad solicitada.

El código debe poder ser ejecutado en SWI-Prolog. No es necesario entregar un informe sobre el trabajo, alcanza con que el código esté adecuadamente comentado. Los objetivos a evaluar en la implementación de los predicados son:

- Corrección.
- Declaratividad.
- Reutilización de predicados previamente definidos.
- Uso de unificación, backtracking, generate and test y reversibilidad de los predicados que correspondan.
- No está permitido usar macros que colapsen cláusulas, como `;` y `->`.
- **Importante:** salvo donde se indique lo contrario, los predicados no deben instanciar soluciones repetidas. Vale aclarar que no es necesario filtrar las soluciones repetidas si la repetición proviene de las características de la entrada.

Importante: se admitirá un único envío, sin excepción alguna. Por favor planifiquen el trabajo para llegar a tiempo con la entrega.

Referencias y sugerencias

Como referencia se recomienda la bibliografía incluida en el sitio de la materia (ver sección *Bibliografía* → *Programación Lógica*).

Se recomienda que, siempre que sea posible, utilicen los predicados ISO y los de SWI-Prolog ya disponibles. Recomendamos especialmente examinar los predicados y metapredicados que figuran en la sección *Útil* de la página de la materia. Pueden hallar la descripción de los mismos en la ayuda de SWI-Prolog (a la que acceden con el predicado `help`). También se puede acceder a la [documentación online de SWI-Prolog](#).

En particular, pueden resultarles útiles los siguientes predicados: `between/3`, `member/2`, `length/2`, `append/3`, `maplist/2`, `maplist/4`, `findall/3`, `not/1`.